

Construindo a Web API Fundacional da MecâniQA

Autor: Levi Falcão de Queiroz
Jadson da Silva Sobrinho
Moisés de Souza Oliveira
Murillo dos Santos Marinho Ferreira
Mateus Queiroz Matos Ferreira

Agenda

O objetivo desta apresentação é demonstrar como o CRUD de Peças e Serviços da MecâniQA foi implementado manualmente em Spring Boot, sem banco de dados e sem @Autowired.

1. Contextualização

o problema da MecâniQA e o objetivo do MVP nesta fase.

2. Conceitos básicos de OO

classes, atributos, modificadores de acesso e construtores.

3. Membros estáticos, Enums e Getters/Setters

geração de código e o catálogo de categorias.

4. Padrão Singleton

como os Repositories mantêm o estado vivo em memória.

5. Controllers e API REST

rotas, verbos e status HTTP das 8 User Stories.

Contextualização e Objetivo do MVP

- A MecâniQA está migrando seu sistema para a Web; a diretoria de TI definiu Java com Spring Boot como stack do backend.
- Objetivo do MVP: CRUD completo de duas entidades de negócio — Peça e Serviço — expostas como API REST, cobrindo as 8 User Stories definidas pelo PO.

Classes, Atributos e Construtores

- Peça e Serviço são classes de modelo: todo atributo é private
- O construtor recebe apenas os dados obrigatórios de negócio e já inicializa o estado interno, como o código único e as datas de criação.
- Em Peça, tamanho e cor ficam de fora do construtor porque a OAT os define como estritamente opcionais, são preenchidos depois via setters.

```
public class Peça {  
  
    private static long proximoCodigo = 1L;  
  
    private Long codigo;  
    private String nome;  
    private String codigoBarras;  
    private String fornecedorMarca;  
    private int quantidadeEstoque;  
  
    public Peça(  
        String nome,  
        String codigoBarras,  
        String fornecedorMarca,  
        int quantidadeEstoque,  
        double precoCusto,  
        double precoVenda,  
        CategoriaPeca categoria  
    ) {  
        this.codigo = proximoCodigo++;  
        this.nome = nome;  
    }  
}
```

Membros Estáticos, Enums e Getters/Setters



- Membros static: `private static long proximoCodigo` pertence à classe, não ao objeto, e por isso é compartilhado por todas as instâncias.
- Enum `CategoriaPeca`: cinco valores fixos, sendo eles `MOTOR`, `SUSPENSAO`, `FREIOS`, `ELETRICA`, `ACESSORIOS`.
- Getters e Setters: getters expõem leitura controlada; setters também chamam `atualizarData()`, mantendo `dataUltimaAtualizacao` sempre coerente.

Controllers e Rotas REST

- @RestController marca a classe como controlador REST;
@RequestMapping("/api/pecas") define o prefixo comum das rotas.
- Cada operação das User Stories virou um endpoint, com o verbo HTTP certo para cada ação.
- O mesmo padrão se repete em /api/servicos, cobrindo as User Stories US05–US08.

```
@RestController
@RequestMapping("/api/pecas")
public class PecaController {

    private final PecaRepository repository;

    public PecaController() {
        this.repository = PecaRepository.getInstance();
    }

    // GET /api/pecas
    // 200 OK
    @GetMapping
    public ResponseEntity<List<Peca>> listar() {

        return ResponseEntity.ok(repository.listar());
    }
}
```

Verbos e Status HTTP



- Implementado com `ResponseEntity<T>`, que permite controlar explicitamente o status de cada resposta. Sem isso, o Spring devolveria sempre 200 OK por padrão.
- Mapeamento usado nos dois Controllers:
 - POST → 201 Created (recurso novo criado)
 - GET / PUT → 200 OK (encontrado ou atualizado)
 - GET / PUT / DELETE → 404 Not Found (código não existe)
 - DELETE → 204 No Content (excluído, sem corpo de resposta)

Padrão Singleton: Por Que e Como



- Sem banco de dados, alguém precisa manter os dados vivos enquanto o servidor roda, e essa é a função dos Repositories.
- O Singleton garante uma única instância do repositório durante toda a execução da aplicação, evitando listas duplicadas e dados inconsistentes.
- Três peças sustentam o padrão:
 - Construtor private
 - Atributo static instance
 - getInstance()

Padrão Singleton: Na Prática



- O Controller nunca cria o repositório, apenas chama `PecaRepository.getInstance()`.
- A lista de peças é atributo de instância do único objeto Singleton, por isso sobrevive entre requisições.

```
public class PecaRepository {  
  
    private static PecaRepository instance;  
  
    private final List<Peca> pecas;  
  
    private PecaRepository() {  
        pecas = new ArrayList<>();  
    }  
  
    public static PecaRepository getInstance() {  
  
        if (instance == null) {  
            instance = new PecaRepository();  
        }  
  
        return instance;  
    }  
}
```

Considerações Finais



- As 8 User Stories foram atendidas com CRUD completo de Peça e Serviço, 100% em memória, sem banco de dados e sem @Autowired.
- O padrão Singleton assumiu o papel que um banco de dados teria nesta fase, mantendo o estado vivo durante toda a execução do servidor.
- Próximo passo natural do produto: substituir os Repositories manuais por Spring Data JPA e um banco real, sem precisar alterar os Controllers.